

### <과제 3 : UDP 기반 ARQ 구현- 202020354 정용훈>

#### 1. 실습 목적 및 개요

이번 과제의 목적은 Stop-and-Wait ARQ 를 이해하고, UDP socket 에서 이를 구현해보면서 timeout sec 과 loss rate (P)를 변화시키면서 이들의 관계를 이해하는 것이다.

#### 2. 데이터 구조 (Syntax)

Client 와 server 간의 데이터 교환 형식을 정의한 msg\_format 을 정의했다. Msg\_format struct 로, 유저로부터 입력받은 메시지인 msg char 배열과, int type 의 seq\_num 을 가진다. 이 struct 는 server 로 전송하기 전, make\_send\_buf 함수에서 하나의 char 배열로 만들어 보내진다.

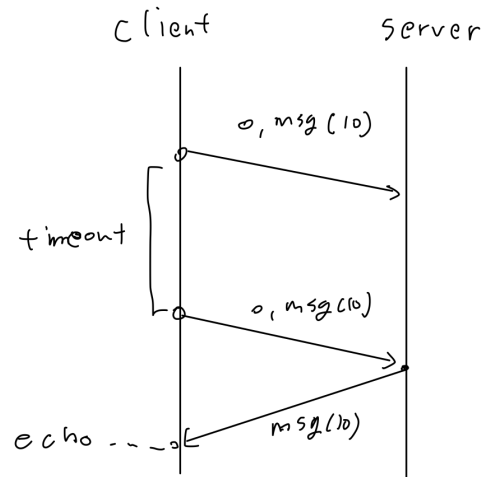
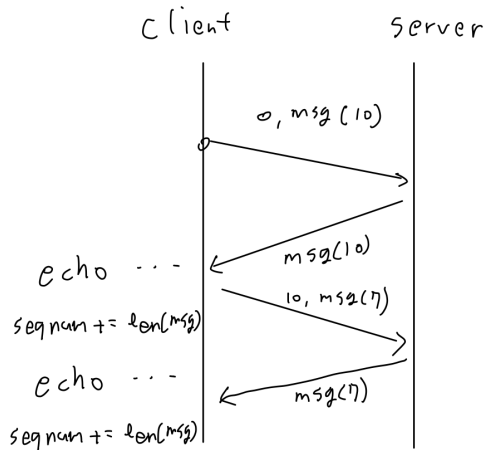
Client 에서는 timeout 시간을 관리하기 위한 int type 의 timeout 변수가 전역에 관리되며, 프로그램 실행 파라미터로 입력받을 경우 입력받은 값으로 설정되며, 아닌 경우 10 초가 default 이다. 또한 quit 상태를 관리하기 위한 int type 의 quit\_flag 는 main 에서 관리되며 실행 시, default 는 0 이다.

Server 에서도 client 와 같은 msg\_format 을 가지며 recv 이후 save\_recv\_buf 함수에서 msg\_format 에 맞게 저장된다. Server 에서는 추가로 client 별 통계치를 나타내기 위한 statistics struct 가 존재하는데, {P, msgs, retry\_msgs, all\_msgs, retry\_rate} 가 각각 {P, N1, N2, N3, R}을 나타낸다.

Server 에서는 여러 client 가 보낼 수 있으므로 종료를 시도한 client 를 관리하기 위한 quit\_sock struct 가 있는데, client 의 주소인 addr, quit\_flag, retry\_flag, statistics 구조체 stats 를 가진다. client 는 처음 메시지를 보내게 되면 quit\_sock 에 등록되고, client 가 종료를 요청하면 quit\_flag 가 1 이 되고 해당 socket 은 종료되게 된다. Quit\_sock 에서 retry\_flag 또한 retry 가 있다면 1 이 되어 retry 를 수행하게 된다.

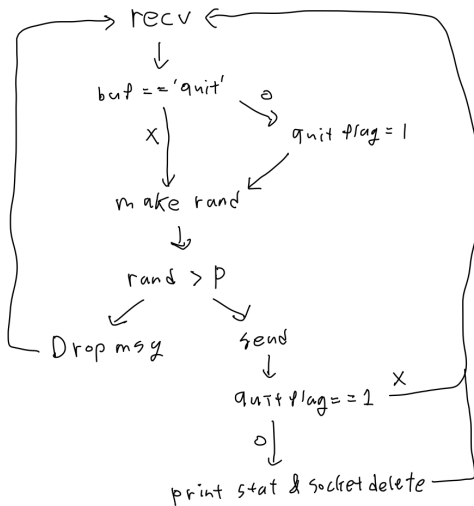
Server 에서도 float type 의 P 는 전역에서 선언되어 관리되며, 프로그램 실행 파라미터로 입력받을 경우 입력받은 값으로 설정되며, 아닌 경우 0.5 가 default 이다. 또한 random number 를 관리하기 위한, float type 의 rand\_num 이 main 에서 관리되고, random 한 100 이하 수를 100 으로 나눠 0~1 사이 랜덤값을 얻는다.

### 3. 프로토콜 동작 (Semantics)



Server 에서 Loss 가 발생한 경우, Client 는 client 가 보낸 메시지는 왼쪽 사진과 같이 다시 msg 를 받아 echo 를 한 후, seq\_num 을 증사시키도록 동작한다.

만약 Server 에서 Loss 가 발생한 경우, Client 는 Timeout 만큼 기다렸다가, 다시 sendto 를 하게 되며 이는 ACK (msg(10)에 대한) 를 받을 때까지 계속 반복된다.



Server 에서는 recv 상태에 있다가, client 로 부터 msg 가 오게 되면 우선 온 msg 가 quit 인지 확인한다. (msg 가 도착하면 quit\_sock 에 client 가 등록되고 quit\_flag 와 retry\_flag 는 0 으로 지정된다.) Quit 이라면 quit flag 를 1 로 설정하고 아닌 경우 random number 를 만들고 rand 와 P 를 비교한다. Rand > P 라면 손실이 발생한 경우로 msg buf 를 clear 하고 다시 recv 에서 대기한다. Rand <= P 인 경우, echo 를 위해 client 로 sendto 하고 만약 quit\_flag 가

1 이라면 stats 를 출력하고 quit\_sock 에서 해당 client 를 지운다. 그리고 다시 recv 에서 다음 메시지를 기다리게 된다.

#### 4. 프로그램 구현

4-1. Client : Client 는 timeout 구현을 위해 이중 while 문으로 구성된다. 첫 번째 반복문에서, 키보드 입력을 기다리고, 유저로부터 입력받은 msg 는 msg\_format 의 msg 에 저장되며 make\_send\_buf 함수를 통해 sendbuf 에 저장한다. Make\_send\_buf 는 msg\_format 에서 seq\_num 을 2 byte 문자로 변환해, send\_buf 에 저장하고 이후 msg 를 이어 붙인다. Seq\_num 을 2byte 안에 모두 담기 위해 unsigned char 타입으로 저장한다. 이후 Inner while 문으로 들어가 send\_buf 를 server 로 전송하고 ACK 를 기다린다. Timeout 을 기다리기 위해, select 를 이용해 timeout 을 지정해준다. 이후 recvfrom 에서 대기하게 되며 Timeout 이내에 받은 메시지가 있다면 정상 동작하며 Inner while 과 Outer while 을 종료하고 프로세스를 종료한다. Timeout 이라면 Inner while 을 반복하여, 다시 timeout 만큼을 기다리게 된다.

4-2. Server : Server 는 bind 이후 client 의 msg 를 recv 하며 recv 된 이후, quit\_sock 에서 관리되고 있지 않은 client 라면 client 의 주소와 flag, stat 을 초기화하여 quit\_sock 에 등록한다. 만약 여기서 quit\_sock 에 등록된 client 라면, 같은 client 가 2 번째 혹은 그 이상 메시지를 보낸 경우로, 다음 메시지를 받은 것으로 처리해야 한다. 이때는 quit\_sock 에서 해당 client 의 stat 값의 받은 메시지수와 all\_msgs 를 증가시켜준다. 이후 받은 msg 를 msg\_format 에 맞게 save\_recv\_buf 함수로 저장하고(client 의 make\_send\_buf 와 반대 과정), 받은 msg 가 quit 인지 strcmp 로 비교한다. ('QUIT' or 'quit') quit 이라면 해당 socket 의 quit\_flag 를 1 로 설정하고 random number 를 계산한다.  $Rand > P$  라면 Loss 가 발생하는 경우로, retry\_flag = 1 로 설정하고 continue 로 while 문의 처음으로 돌아가 recv 한다. Retry\_flag 는 recv 이후 stats 의 msg 와 all\_msgs 를 증가시킬때, retry 의 경우 이미 받았던 메시지이므로 증가시키지 않기 위해 사용된다. 또한 retry\_flag 를 1 로 설정할때, quit\_flag 가 0 이라면 종료가 아닌 경우로 stats 의 retry\_msgs 를 증가시켜준다.  $Rand \leq P$  라면 echo case 로, client 로 send 를 수행하고 이 때, retry\_msgs 와 all\_msgs 를 이용해 retry\_rate 를 계산한다. 이후 quit\_flag 를 확인해 종료해야 하는 상태라면 stats 의 통계량을 출력하고 quit\_sock 에서 해당 client 의 socket 의 메모리를 초기화한다.

## 5. 결과

```
(base) jeong-yonghun@Huns-MacBook-Pro Assign3_UDP_ARQ % ./client 127.0.0.1 9001 1
```

```
IP : 127.0.0.1
```

```
Port : 9001
```

```
[Input Message] hi there?
```

```
[ROW DATA]
```

```
[SEND MSG]          hi there?
```

```
[UDP Client]        Sent 12 bytes.
```

```
[SEQ NUM]           0
```

```
[ ... WAITING ... ]
```

```
[TIMEOUT]           Retransmitting...
```

```
[UDP Client]        Sent 12 bytes.
```

```
[SEQ NUM]           0
```

```
[ ... WAITING ... ]
```

```
[TIMEOUT]           Retransmitting...
```

```
[UDP Client]        Sent 12 bytes.
```

```
[SEQ NUM]           0
```

```
[ ... WAITING ... ]
```

```
[UDP Client]        Received 12 bytes.
```

```
[Received Message]  hi there?
```

```
[ ..... DONE ..... ]
```

```
[Input Message] what's up?
```

```
[ROW DATA]
```

```
[SEND MSG]          what's up?
```

```
[UDP Client]        Sent 13 bytes.
```

```
[SEQ NUM]           9
```

```
[ ... WAITING ... ]
```

```
[TIMEOUT]           Retransmitting...
```

```
[UDP Client]        Sent 13 bytes.
```

```
[SEQ NUM]           9
```

```
[ ... WAITING ... ]
```

```
[UDP Client]        Received 13 bytes.
```

```
[Received Message]  what's up?
```

```
[ ..... DONE ..... ]
```

[Input Message] im good

[ROW DATA]

[SEND MSG] im good

[UDP Client] Sent 10 bytes.

[SEQ NUM] 19

[ ... WAITING ... ]

[UDP Client] Received 10 bytes.

[Received Message] im good

[ ..... DONE ..... ]

[ROW DATA]

[RECV MSG] hi there?

[UDP/127.0.0.1:58535] hi there?

[RAND SEED] 0.65

[DROP MSG] [ ... XXX ... ]

[ROW DATA]

[RECV MSG] hi there?

[UDP/127.0.0.1:58535] hi there?

[RAND SEED] 0.92

[DROP MSG] [ ... XXX ... ]

[ROW DATA]

[RECV MSG] hi there?

[UDP/127.0.0.1:58535] hi there?

[RAND SEED] 0.42

[ACK ABOUT] 0

[ROW DATA]

[RECV MSG] what's up?

[UDP/127.0.0.1:58535] what's up?

[RAND SEED] 0.87

[DROP MSG] [ ... XXX ... ]

[ROW DATA]

[RECV MSG] what's up?

[UDP/127.0.0.1:58535] what's up?

[RAND SEED] 0.03

[ACK ABOUT] 9

[ROW DATA]

[RECV MSG] im good

[UDP/127.0.0.1:58535] im good

[RAND SEED] 0.27

[ACK ABOUT] 19

위의 첫 번째, 두 번째 사진은 client, 세 번째는 server 이다. Client 가 server 로 “hi there?”을 보내게 되면 msg 는 9byte 로 seq\_num 2 byte 와 null 1byte 를 포함해 13 byte 를 보내게 되고 이 경우에는 server 의 rand\_num 이 0.65, 0.92, 0.42 로 0.5 이상인 첫 번째, 두 번째에서 retransmission 이 일어났고, 세 번째에 echo 를 수행한 것을 볼 수 있다.

그리고 client 가 “what’s up?”을 보내게 되면 이전 msg 가 9byte 로 seq\_num 가 9 로 증가한 것을 볼 수 있고, 이 경우에도 server 의 random number 가 0.87, 0.03 으로 첫 번째에서 Loss 가 생겼고 두 번째에 echo 된 것을 볼 수 있다.

다음으로 Client 가 “im good”을 보냈을 때, 이전 msg 크기만큼(10 byte) seq\_num 가 증가한 것을 확인할 수 있고, server 의 rand\_num 가 0.27 로 바로 echo 를 수행한 것을 확인할 수 있다.

```
[Input Message] quit

[ROW DATA]
[SEND MSG]      quit
[UDP Client]    Sent 7 bytes.
[SEQ NUM]       26
                [ ... WAITING ... ]

[TIMEOUT]       Retransmitting...
[UDP Client]    Sent 7 bytes.
[SEQ NUM]       26
                [ ... WAITING ... ]

[UDP Client]    Received 7 bytes.
[Received Message] quit
                [ ..... DONE .... ]
(base) jeong-yonghun@Huns-MacBook-Pro Assign3_UDP_ARQ %
```

```
[ROW DATA]
[RECV MSG]      quit
[RAND SEED]     0.72
[DROP MSG]      [ ... XXX ... ]

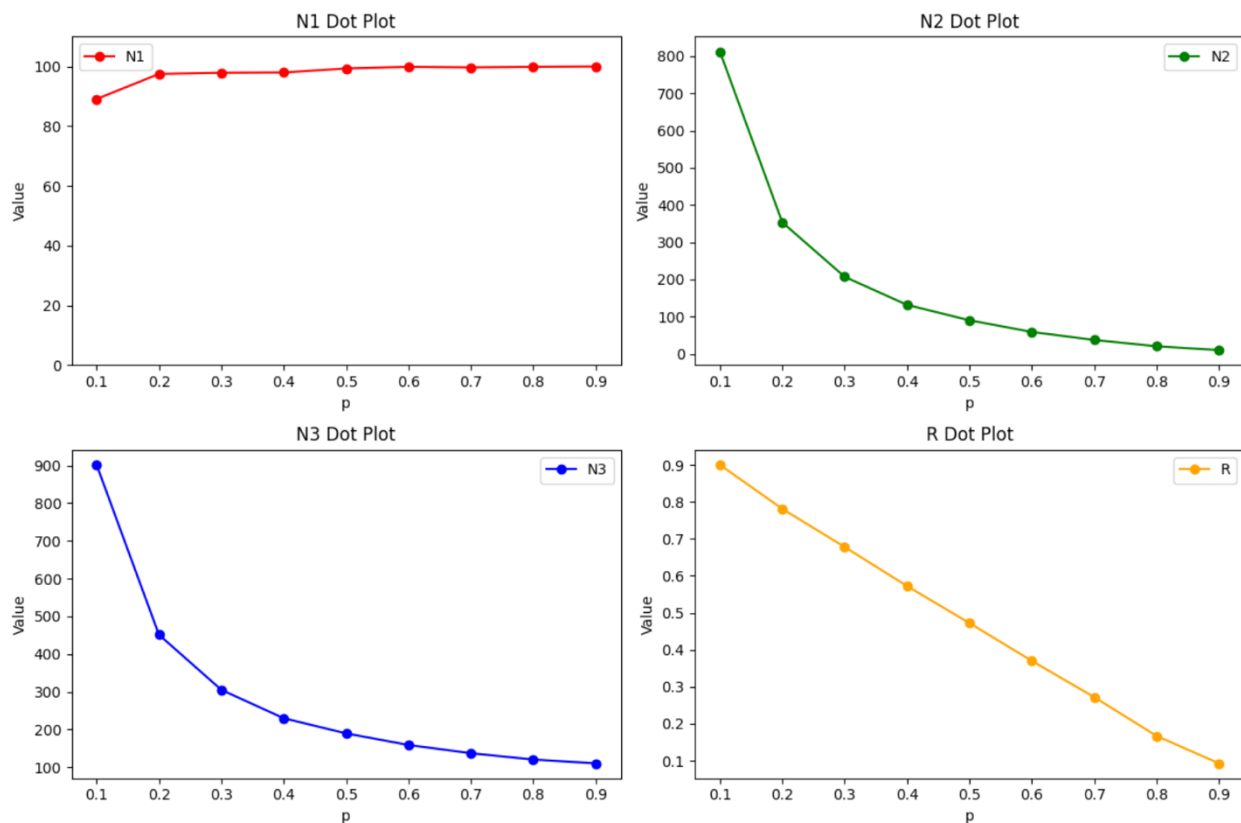
[ROW DATA]
[RECV MSG]      quit
[RAND SEED]     0.44
[ACK ABOUT]     26

[QUIT/127.0.0.1:61580] [REMOVED]
[STATISTICS]    p=<0.50>, N1=<2>, N2=<2>, N3=<4>, R=<0.50>
```

첫 번째 사진은 client 의 터미널, 두 번째 사진은 server 의 터미널이다. Client 가 quit 을 보냈을 때, quit msg 도 rand\_num 이  $0.72 > 0.5$  로 Loss 가 생겨 한번 retransmission 이 일어난 것을 볼 수 있다. 이후 client 가 종료되고 통계값을 확인할 수 있다. 해당 client 와 주고받은 msg 는 총 3 개, retransmission 은 총 3 번 일어나 retry\_rate 는 0.5 인 것을 확인할 수 있다.

통계 작성을 위한 test code 는 UDP\_ARQ\_client\_test.c 와 UDP\_ARQ\_server\_test.c 를 이용했다. UDP\_ARQ\_client\_test.c 는 키보드 입력을 받지 않고 random 6 자리 숫자를 server 에게 보내며 총 100 번 보내고 timeout 은 1 초로 동작한다. UDP\_ARQ\_server\_test.c 는 기존의 server 코드와 동일하나 print 문을 모두 제거했고, 마지막 통계치를 출력하는 것이 아니라 data.tsv 에 쓰도록 수정한 버전이다. 테스트 코드는 첨부한 Makefile 로 test\_script.sh 를 실행하여 (make run\_tests) 테스트를 진행했으며, p 가 0~1 까지 0.1 씩 증가하고 하나의 p 에 대해 client 를 10 회 반복하여 테스트를 진행했다.

저장한 data.tsv 는 python pandas 와 matplotlib 을 이용해 그래프로 그렸으며, evaluate.ipynb 에서 진행했다.



위 사진은 통계량을 matplotlib 을 이용해 그린 그래프이다. N1 의 경우 모두 100 번 전송했는데 어떤 이유인지는 모르겠으나, 일부 손실이 발생한 것을 볼 수 있다. N2, N3 의 경우 0.1 에서 큰 값을 가지고 이후 비선형적으로 감소하는 것을 볼 수 있다. R 의 경우 P 가 증가함에 따라 linear 하게 감소하는 것을 확인할 수 있다.

$$p(L) : \text{Loss 확률} \quad p(L) = 1-p$$

$$p(E) : \text{echo 확률} \quad p(E) = p$$

$$\text{try 1 에 성공 확률} : p$$

$$\text{try 2 에 성공 확률} : (1-p)p$$

$$\text{try 3 에 성공 확률} : (1-p)^2 p$$

$$\begin{aligned} \text{평균 시도 횟수의 기대값 } E(X) \\ E(X) &= \sum_{k=1}^{\infty} k (1-p)^{k-1} p \\ &= E_1(X) p \\ &= \frac{1}{p^2} p \\ &= \frac{1}{p} \end{aligned} \quad \left\{ \begin{aligned} E_1(X) &= \sum_{k=1}^{\infty} k (1-p)^{k-1}, (q=1-p) \\ E_1(X) &= \sum_{k=1}^{\infty} k q^{k-1}, (q < 1) \\ E_1(X) &= \frac{1}{(1-q)^2} = \frac{1}{(1-(1-p))^2} = \frac{1}{p^2} \end{aligned} \right.$$

$$\therefore (\text{재전송을 위한 평균 시도 횟수}) = \frac{1}{p}$$

평균 시도 횟수에는 첫 번째 전송 포함.

$$(\text{retry 기대값}) = \frac{1}{p} - 1$$

$$N_2 = N_1 \left( \frac{1}{p} - 1 \right)$$

$$\begin{aligned} N_3 &= N_1 + N_2 \\ &= N_1 + N_1 \left( \frac{1}{p} - 1 \right) \\ &= \frac{N_1}{p} \end{aligned}$$

$$N_3 = \frac{100}{p} \quad (N_1 = 100)$$

위와 같이 N3 를 P 에 대한 식으로 유도할 수 있다. 아래 그래프는 유도된  $y = 100/x$  와 측정된 P 에 대한 N3 를 같이 그려본 결과 거의 유사한 것을 확인할 수 있다. 실제

테스트에서도  $p$  가 0 일 때는 무한히 retransmission 을 하여 0 을 제외하고 0.1 부터 시작했다. 구해진 공식에 의해서도  $p$  가 0 일 때는 무한대임을 알수 있다.

